

Report - EvoAgent Advanced NLP06 Assignment 03

1. Team, Goal, and Final Result

This project implements an EvoAgent-style self-improving system for Vietnamese financial question answering. The model must produce numeric answers by generating and executing a small DSL over assignment-provided text/table context.

Team:

Member	Student ID	Class	Kaggle ID	Main contribution
Melanie	TBD	Advanced NLP06	yeyeezyzeus	Qwen fine-tuning, rejection sampling/STaR, API ensemble, final high-score system.
Nguyen Phan Nguyen - Stephen	TBD	Advanced NLP06	nguynpnphannguy	EvoAgent Stages 0-4, ARC runs, early hybrid/retry experiments, packaging and documentation.

Designated ThinkFlic submitter: TBD.

Video presentation link: TBD Google Drive link with "Anyone with the link can view" access.

Final Kaggle artifact:

- File in package: kaggle/final_submission.csv
- Submitted filename: submission_ensemble_api3.csv
- Public leaderboard score: 0.80161
- Strongest self-hosted alternate: submission_ft_qwen3_8b_rs.csv, public score 0.74696

2. EvoAgent Implementation and Milestone 1

The core EvoAgent implementation completed the required staged pipeline:

- Stage 0: sandbox baseline prediction and proof generation.
- Stage 1: DSL/program execution and token accounting.
- Stage 2: reflection over failed examples and failure modes.
- Stage 3: proposal of new strategies with validation.
- Stage 4: evolutionary harness with parent selection, smoke tests, history tracking, and proof output.

Local graders passed:

```
python3 graders/grade_stage0.py
python3 graders/grade_stage1_executor.py
python3 graders/grade_stage2_reflector.py
python3 graders/grade_stage3_proposer.py
PYTHONPATH=. python3 graders/grade_smoke_proof.py
python3 graders/grade_stage4_harness.py
```

The ARC evolution run produced a best dev accuracy of 0.48333333333333334 from a 0.42 baseline, with best iteration 1. The required evidence artifacts included in this package are evolution_proof.json, failure_mode_report.pdf, learning_curve.pdf, and strategy_diversity.pdf.

3. Data, Integrity, and Allowed Inputs

Training and validation used only the provided assignment data:

- data/train.json
- data/validation.json
- data/test.json
- generated predictions, programs, and details from documented model runs

No hidden labels, manual test labeling, source PDF lookup, filename/page metadata lookup, external web search, or leaked answers were used. API models were used only as solvers over the same assignment-provided row context visible to the open models. API keys, Hugging Face tokens, Kaggle tokens, model weights, caches, and private keys are excluded from the package.

4. Final System

The final system is a deterministic three-way ensemble:

1. Rejection-sampled fine-tuned Qwen3-8B predictions.
2. Gemini 2.5 Flash predictions.
3. DeepSeek-V3 predictions.

Each member generated DSL-style answers from the provided question and context.

The final answer was selected by deterministic voting, with ties broken toward the rejection-sampled Qwen3-8B model. This voting rule was chosen because the three models had different error patterns. The hosted API models were not used to retrieve external evidence; they only processed assignment-provided context.

The best self-hosted path was Qwen3-8B fine-tuned with QLoRA, then improved with rejection sampling / STaR. The training data used the assignment training programs in the same prompt format used for inference. Rejection sampling drew multiple candidate programs per training question, executed them with the DSL evaluator, kept distinct programs whose value matched the gold answer, and retrained on the augmented verified program set.

Dev-slice evidence from the fine-tuning track:

Slice	Prompted Qwen3-8B	Fine-tuned Qwen3-8B	RS / STaR
--- ---	--- ---	--- ---	
Overall	23.3%	78.1%	77.9% value-mode / 79.6% program-mode
Arithmetic	15.5%	76.3%	78.0%
Table lookup	59.6%	87.2%	88.3%
Ratio/percent	18.4%	80.4%	not separately recorded
Multi-step, 3+ ops	4.3%	47.8%	60.9%

Figures included in evidence/:

- results_progression.png: public-score progression across Phase 3.
- results_star_slices.png: fine-tuned vs STaR dev-slice comparison.

5. Kaggle Experiments and Selection

Early EvoAgent and hybrid work established the baseline:

Run	Method	Public score	Decision
Run001	EvoAgent ARC best strategy	0.56477	Baseline
Run002	iter003 table-op strategy	0.47975	Useful fallback only
Run003	Run001 fallback to Run002 nonzero	0.64574	First major gain
Run005	Broad numeric post-processing	0.64170	Rejected
Run008	Targeted retry over zero rows	0.65587	Superseded
Run009-lite	Safe suspicious-row retry	0.65789	Stephen-only best
Run012	GPT-OSS safe Three GPT-OSS corrections over Run009	0.66194	Superseded

Team fine-tuning and ensembling produced the final gains:

Submission	Public score	Notes
submission_ensemble.csv	0.69433	First diverse ensemble
submission_ensemble_v2.csv	0.69838	Member-confirmed scale fixes
submission_ensemble_v4.csv	0.70242	Member-confirmed sign fixes
submission_ft_qwen3_8b.csv	0.72267	Fine-tuned Qwen3-8B
submission_ft_qwen25_7b.csv	0.71255	Fine-tuned Qwen2.5-7B diversity model
submission_ft_merged.csv	0.72874	Broken-row fallback merge
submission_ft_qwen3_8b_rs.csv	0.74696	Best self-hosted STaR model
submission_ensemble_api3.csv	0.80161	Primary final

Negative results shaped the final choice. Broad numeric post-processing reduced score, same-family ensembles were flat or regressive, and from-scratch prompted members produced low dev accuracy despite occasional table-lookup improvements. The winning pattern was not broad correction; it was stronger specialized models plus genuinely diverse ensemble members.

6. Reproducibility, Compute, and Cost

Core reproduction steps are in `source_code/run_instructions.md`. The package includes `source_code/src/` plus Phase 3 scripts for prediction generation, self-consistency, SFT dataset construction, QLoRA training, LoRA merging, rejection sampling, dev evaluation, and deterministic ensemble voting.

The available final-branch scripts are stored in `source_code/phase3_scripts/`. The exact API generation script referenced by the final report was not present on `melanie-evoagent-arc` at packaging time, so the saved API prediction artifacts are included in `evidence/prediction_artifacts/`. If that script is recovered, the API run can be reproduced with externally supplied `GEMINI_API_KEY` and `DEEPSEEK_API_KEY` environment variables.

Compute summary:

- EvoAgent and early Kaggle runs used VT ARC GPU infrastructure.
- Qwen3-8B fine-tuning used QLoRA on an A100/H100-class 80GB GPU.
- Rejection sampling used GPU inference plus local DSL execution as verifier.
- API predictions used Gemini 2.5 Flash and DeepSeek-V3 with deterministic prompts over provided context only.
- Approximate API/GPU cost was modest, but exact final dollar total is TBD.

Repository commit documented for this package: b1b3aa9d7dfb1ce33358766349a3d97090ae80f9.

7. Limitations and Lessons Learned

The final API3 score demonstrates the reachable ceiling, but the strongest fully self-hosted result remains 0.74696. The main future direction is to close the gap using the ≤ 9 B self-hosted model through further STaR rounds or reinforcement learning from verifiable DSL rewards. The task is well suited to that because the DSL evaluator provides a checkable reward signal.

The most important lesson is that specialization and verification beat raw prompting. Fine-tuning taught the exact DSL dialect and answer conventions; rejection sampling used execution as a correctness filter; and the final ensemble worked because its members were genuinely diverse. Broad post-processing and same-family ensembles were less reliable and were rejected when validation or leaderboard feedback showed regression.